

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Департамент программной инженерии

СОГЛАСОВАНО

УТВЕРЖДАЮ

Профессор департамента программной инженерии ФКН НИУ ВШЭ

Академический руководитель образовательной программы «Программная инженерия»

« ____ » _____ 2026 г.

« ____ » _____ 2026 г.

ГЕНЕРАТОР ДАННЫХ ДЛЯ АНАЛИТИЧЕСКОГО
БЕНЧМАРКА СУБД ТРС-Н

Текст программы

Лист УТВЕРЖДЕНИЯ

RU.17701729.11.03-01 12 01-1-ЛУ

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл	

Исполнитель: Студент БПИ225

« ____ » _____ 2026 г.

УТВЕРЖДЁН
RU.17701729.11.03-01 12 01-1-ЛУ

ГЕНЕРАТОР ДАННЫХ ДЛЯ АНАЛИТИЧЕСКОГО
БЕНЧМАРКА СУБД ТРС-Н

Текст программы

RU.17701729.11.03-01 12 01-1

Листов 19

Инв. № подл	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Содержание

1	Символическая запись на исходном языке	3
1.1	Модуль регистрации таблиц	3
1.2	Модуль генерации типовых значений	3
1.3	Модуль формирования пула текста	5
1.4	Модуль выбора стратегии записи	6
1.5	Модуль записи в формат Parquet	6
1.6	Модуль записи в формат ORC	7
1.7	Модуль записи в формат Lance	8
1.8	Модуль записи в формат Vortex	9
2	Символическая запись алгоритмов формирования таблиц	11
2.1	Формирование таблицы region	11
2.2	Формирование таблицы nation	11
2.3	Формирование таблицы supplier	12
2.4	Формирование таблицы part	12
2.5	Формирование таблицы partsupp	13
2.6	Формирование таблицы customer	14
2.7	Формирование таблицы orders	14
2.8	Формирование таблицы lineitem	16
3	Символическое представление машинных кодов	18

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. Символическая запись на исходном языке

В настоящем разделе приводится символическая запись основных модулей программы, определяющих состав поддерживаемых таблиц, способы генерации типовых значений и порядок записи сформированных данных в выходные файлы.

1.1. Модуль регистрации таблиц

Ниже приведена символическая запись модуля, задающего состав таблиц, поддерживаемых программой, и связывающего их с соответствующими генераторами строк и схемами колонок.

```
func register_tables():
    registry.add("region",    region_schema,    region_generator)
    registry.add("nation",    nation_schema,    nation_generator)
    registry.add("supplier",  supplier_schema,  supplier_generator)
    registry.add("part",      part_schema,      part_generator)
    registry.add("partsupp",  partsupp_schema, partsupp_generator)
    registry.add("customer",  customer_schema,  customer_generator)
    registry.add("orders",    orders_schema,    orders_generator)
    registry.add("lineitem",  lineitem_schema,  lineitem_generator)
```

Листинг 1 — Регистрация таблиц генератора

1.2. Модуль генерации типовых значений

Ниже приведены символические записи основных компонентов модуля генерации типовых значений. Отдельно отражены генерация ограниченных числовых значений, выбор токенов из распределений, построение строковых представлений и преобразование дат ТРС-Н.

```
func next_bounded_int(seed, lower_bound, upper_bound, values_per_row):
    random = RowRandomInt(seed, values_per_row)
    return random.next_int(lower_bound, upper_bound)

func next_bounded_long(seed, lower_bound, upper_bound, values_per_row):
    random = RowRandomLong(seed, values_per_row)
    return random.next_long(lower_bound, upper_bound)

func next_bounded_value(seed, lower_bound, upper_bound, scale_factor, values_per_row):
    use_64bits = scale_factor >= 30000 or upper_bound > INT32_MAX

    if use_64bits:
        return next_bounded_long(seed, lower_bound, upper_bound, values_per_row)

    return next_bounded_int(seed, lower_bound, upper_bound, values_per_row)

func advance_rows(random, row_count):
    random.advance_rows(row_count)

func finish_row(random):
    random.row_finished()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 2 — Генерация ограниченных числовых значений

```

func next_token(distribution, random):
    sample = random.next_int(0, distribution.total_weight - 1)
    cumulative = 0

    for entry in distribution:
        cumulative += entry.weight
        if sample < cumulative:
            return entry.token

    return distribution.last.token

func next_string(distribution, random):
    return next_token(distribution, random)

func next_string_sequence(distribution, count, random):
    values = copy_all_tokens(distribution)

    for current in range(0, count):
        swap_with = random.next_int(current, len(values) - 1)
        swap(values[current], values[swap_with])

    return join_with_spaces(values[0:count])

```

Листинг 3 — Генерация строк из распределений

```

func next_alphanumeric(average_length, random):
    length = random.next_int(0.4 * average_length, 1.6 * average_length)
    text = ""

    while len(text) < length:
        chunk = random.next_int(0, INT32_MAX)
        text += decode_base64_like_alphabet(chunk)

    return text[0:length]

func next_phone(nation_key, random):
    country_code = 10 + (nation_key mod 90)
    local1 = random.next_int(100, 999)
    local2 = random.next_int(100, 999)
    local3 = random.next_int(1000, 9999)
    return format("%02d-%03d-%03d-%04d", country_code, local1, local2, local3)

```

Листинг 4 — Генерация символьных и телефонных значений

```

func to_unix_epoch(generated_date):
    return (generated_date - MIN_GENERATE_DATE) + UNIX_EPOCH_OFFSET

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

func is_in_past(generated_date):
    return to_julian(generated_date) <= CURRENT_DATE

func format_tpch_date(generated_date):
    year, month, day = to_ymd(generated_date)
    return format("%04d-%02d-%02d", year, month, day)

```

Листинг 5 — Преобразование дат TPC-H

1.3. Модуль формирования пула текста

Ниже приведена символическая запись модуля `TextPool`, используемого для построения текстовых атрибутов. Пул заполняется синтетическими предложениями, построенными по грамматическим шаблонам TPC-H, после чего генераторы таблиц через компонент `RandomText` берут из него подстроки без дополнительного конструирования текста для каждой строки.

```

func build_text_pool(target_size):
    random = RowRandomInt(933588178, MAX_INT)
    text = ""

    while len(text) < target_size:
        text += generate_sentence(random)

    return text[0:target_size]

func generate_sentence(random):
    sentence = ""
    syntax = next_token(grammar_distribution, random)

    for symbol in syntax:
        if symbol == "V":
            sentence += generate_verb_phrase(random)
        elif symbol == "N":
            sentence += generate_noun_phrase(random)
        elif symbol == "P":
            sentence += next_token(prepositions, random) + " the "
            sentence += generate_noun_phrase(random)
        elif symbol == "T":
            sentence = trim_trailing_space(sentence)
            sentence += next_token(terminators, random)

    sentence += " "

    return sentence

func next_text(text_pool, average_length, random):
    min_length = 0.4 * average_length
    max_length = 1.6 * average_length

    offset = random.next_int(0, text_pool.size - max_length)
    length = random.next_int(min_length, max_length)

    /* Возврат представления */

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
return text_pool.view(offset, offset + length)
```

Листинг 6 — Формирование и использование TextPool

1.4. Модуль выбора стратегии записи

Ниже приведена символическая запись модуля, определяющего способ записи сформированных данных. Для форматов Parquet и ORC поддерживаются стратегии `parallel-files` и `single-file-ordered`; для форматов Lance и Vortex используется только упорядоченная запись в один логический набор файлов.

```
func generate_table_with_strategy(context, table, output, writer_options, format_driver,
    scheduler_options):
    part_count = format_driver.resolve_part_count(table, context.scale, writer_options)
    strategy = resolve_selected_write_strategy(scheduler_options, format_driver)

    if strategy == "parallel-files":
        return run_parallel_partition_files(
            context, table, output, writer_options, format_driver, part_count
        )

    if format_driver.ordered_execution_model() == "native-multiplexed":
        return run_native_multiplexed_ordered(
            context, table, output, writer_options, format_driver, part_count
        )

    return run_foreign_streaming_ordered(
        context, table, output, writer_options, format_driver, part_count
    )

func resolve_selected_write_strategy(options, format_driver):
    if options.write_strategy == "auto":
        strategy = format_driver.preferred_strategy()
    else:
        strategy = options.write_strategy

    assert format_driver.supports_strategy(strategy)
    return strategy
```

Листинг 7 — Выбор стратегии записи

1.5. Модуль записи в формат Parquet

Ниже приведена символическая запись модуля записи в формат Parquet. В зависимости от выбранной стратегии данные либо распределяются по нескольким файлам партиций, либо несколькими производителями готовятся параллельно и затем в порядке номеров партиций объединяются в один файл `table-1.parquet`.

```
func parquet_parallel_partition_files(table, partition, batches, output, options):
    path = output / table.name / format_parquet_partition_name(table, partition)
    temp_path = path + ".tmp"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

sink = open_output_stream(temp_path)
sink = maybe_buffer(sink, options.output_buffer_bytes)

writer = open_parquet_writer(
    schema = table.schema,
    compression = options.compression,
    row_group_rows = resolve_row_group_rows(table, options),
    use_threads = options.use_threads
)

for batch in batches:
    writer.write_record_batch(batch)

writer.close()
sink.close()
move_file(temp_path, path)

func parquet_single_file_ordered(table, output, options, part_count):
    sink = open_single_output(output / table.name / (table.name + "-1.parquet"))
    producers = spawn_partition_generators(part_count)
    states = {}
    expected_part = 1

    while message = read_batch_message(producers):
        states[message.part].append(message)

        while states[expected_part].is_ready():
            /* Партиция равна row group */
            sink.begin_partition()

            for batch in states[expected_part].batches:
                sink.write_batch(batch)

            sink.end_partition()
            expected_part += 1

    sink.close()

```

Листинг 8 — Запись таблицы в формат Parquet

1.6. Модуль записи в формат ORC

Ниже приведена символическая запись модуля записи в формат ORC. При стратегии `parallel-files` каждая партиция записывается в собственный файл, а при стратегии `single-file-ordered` батчи партиций последовательно направляются в единый ORC writer с сохранением порядка партиций.

```

func orc_parallel_partition_files(table, partition, batches, output, options):
    path = output / table.name / format_orc_partition_name(table, partition)
    temp_path = path + ".tmp"

    sink = open_output_stream(temp_path)
    sink = maybe_buffer(sink, options.output_buffer_bytes)

    writer = open_orc_writer(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        schema = table.schema,
        compression = options.compression,
        stripe_bytes = resolve_stripe_bytes(table, options)
    )

    for batch in batches:
        writer.write(batch)

    writer.close()
    sink.close()
    move_file(temp_path, path)

func orc_single_file_ordered(table, output, options, part_count):
    sink = open_single_output(output / table.name / (table.name + "-1.orc"))
    producers = spawn_partition_generators(part_count)
    states = {}
    expected_part = 1

    while message = read_batch_message(producers):
        states[message.part].append(message)

        while states[expected_part].is_ready():
            /* Явного начала stripe нет */
            sink.begin_partition()

            for batch in states[expected_part].batches:
                sink.write_batch(batch)

            sink.end_partition()
            expected_part += 1

    sink.close()

```

Листинг 9 — Запись таблицы в формат ORC

1.7. Модуль записи в формат Lance

Ниже приведена символическая запись модуля записи в формат Lance. Для данного формата используется только стратегия **single-file-ordered**: генератор последовательно подает партии в упорядоченный writer, а фактическая запись набора данных выполняется через мост в реализацию на Rust.

```

func lance_single_file_ordered(table, output, options, part_count):
    dataset_path = output / table.name / (table.name + "-1.lance")
    schema = export_arrow_schema(table.schema)
    bridge_options = build_lance_bridge_options(options)

    /* Вызов Rust FFI */
    handle = lance_writer_open(dataset_path, schema, bridge_options)

    for part in range(1, part_count + 1):
        /* Вызов Rust FFI */
        lance_writer_begin_partition(handle, part, part_count)

        for batch in generate_partition_batches(table, part):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    if batch.row_count == 0:
        continue

    arrow_batch = export_record_batch(batch)

    /* Вызов Rust FFI */
    lance_writer_push_batch(handle, arrow_batch)

    /* Вызов Rust FFI */
    lance_writer_end_partition(handle)

/* Вызов Rust FFI */
lance_writer_finish(handle)
lance_writer_destroy(handle)

```

Листинг 10 — Запись таблицы в формат Lance

1.8. Модуль записи в формат Vortex

Ниже приведена символическая запись модуля записи в формат *Vortex*. Как и в случае *Lance*, запись выполняется только в режиме *single-file-ordered*; сериализация батчей и финализация файла осуществляются через мост в реализацию на Rust.

```

func vortex_single_file_ordered(table, output, options, part_count):
    file_path = output / table.name / (table.name + "-1.vortex")
    schema = export_arrow_schema(table.schema)
    bridge_options = build_vortex_bridge_options(options)
    wrote_non_empty_batch = false

    /* Вызов Rust FFI */
    handle = vortex_writer_open(file_path, schema, bridge_options)

    for part in range(1, part_count + 1):
        /* Вызов Rust FFI */
        vortex_writer_begin_partition(handle, part, part_count)

        for batch in generate_partition_batches(table, part):
            if batch.row_count == 0:
                continue

            arrow_batch = export_record_batch(batch)

            /* Вызов Rust FFI */
            vortex_writer_push_batch(handle, arrow_batch)
            wrote_non_empty_batch = true

            /* Вызов Rust FFI */
            vortex_writer_end_partition(handle)

        if not wrote_non_empty_batch:
            empty_batch = make_empty_record_batch(table.schema)

            /* Вызов Rust FFI */
            vortex_writer_push_batch(handle, export_record_batch(empty_batch))

    /* Вызов Rust FFI */

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
vortex_writer_finish(handle)
vortex_writer_destroy(handle)
```

Листинг 11 — Запись таблицы в формат Vortex

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. Символическая запись алгоритмов формирования таблиц

В настоящем разделе приводится символическая запись алгоритмов генерации таблиц бенч-марка ТРС-Н. Для каждой таблицы отражены правила формирования ключей, способы вычисления зависимых атрибутов и использование детерминированных последовательностей случайных значений.

2.1. Формирование таблицы region

Ниже приведена символическая запись алгоритма формирования таблицы **region**. Таблица имеет фиксированный размер, а ее названия берутся из предопределенного справочника; случайным образом формируются только текстовые комментарии.

```
func generate_region(partition_range, text_pool):
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)
    advance_randoms(comment_random, partition_range.start_row)

    end_row = min(partition_range.end_row, REGION_COUNT)

    for row_id in range(partition_range.start_row, end_row):
        emit RegionRow(
            regionkey = row_id,
            name = region_names[row_id],
            comment = comment_random.next_value()
        )

    comment_random.row_finished()
```

Листинг 12 — Формирование таблицы region

2.2. Формирование таблицы nation

Ниже приведена символическая запись алгоритма формирования таблицы **nation**. Наименование государства берется из предопределенного набора, а ссылка на регион вычисляется по накопленной сумме весов, заданных в справочнике наций.

```
func generate_nation(partition_range, text_pool):
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)
    region_cum_sum = sum(nations[i].weight for i in range(0, partition_range.start_row))

    advance_randoms(comment_random, partition_range.start_row)
    end_row = min(partition_range.end_row, NATION_COUNT)

    for row_id in range(partition_range.start_row, end_row):
        nation = nations[row_id]
        region_cum_sum += nation.weight

        emit NationRow(
            nationkey = row_id,
            name = nation.name,
            regionkey = region_cum_sum,
            comment = comment_random.next_value()
        )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
comment_random.row_finished()
```

Листинг 13 — Формирование таблицы nation

2.3. Формирование таблицы supplier

Ниже приведена символическая запись алгоритма формирования таблицы **supplier**. Имя поставщика строится по ключу, адрес и финансовые поля генерируются случайным образом, а в комментарий при выполнении условия может встраиваться специальная подстрока, соответствующая шаблону жалобы или рекомендации клиента.

```
func generate_supplier(partition_range, text_pool):
    address_random = RandomAlphaNumeric(address_seed, average_address_length)
    nation_random = RandomBoundedInt(nation_seed, 0, NATION_COUNT - 1)
    phone_random = RandomPhoneNumber(phone_seed)
    acctbal_random = RandomBoundedInt(acctbal_seed, acctbal_min, acctbal_max)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)
    bbb_gate_random = RandomBoundedInt(bbb_seed, 1, SCALE_BASE)
    bbb_junk_random = RowRandomInt(bbb_junk_seed, 1)
    bbb_offset_random = RowRandomInt(bbb_offset_seed, 1)
    bbb_type_random = RandomBoundedInt(bbb_type_seed, 0, 100)

    advance_all_randoms_to(partition_range.start_row)

    for row_id in range(partition_range.start_row, partition_range.end_row):
        supplier_key = row_id + 1
        nation_key = nation_random.next_value()
        comment = copy(comment_random.next_value())

        if bbb_gate_random.next_value() <= BBB_COMMENTS_PER_SCALE_BASE:
            comment = apply_bbb_comment(comment, bbb_junk_random, bbb_offset_random,
            bbb_type_random)

        emit SupplierRow(
            supkey = supplier_key,
            name = format("Supplier#%09d", supplier_key),
            address = address_random.next_value(),
            nationkey = nation_key,
            phone = phone_random.next_value(nation_key),
            acctbal = acctbal_random.next_value() / 100.0,
            comment = comment
        )

    finish_all_randoms_for_row()
```

Листинг 14 — Формирование таблицы supplier

2.4. Формирование таблицы part

Ниже приведена символическая запись алгоритма формирования таблицы **part**. Имя детали составляется из перестановки токенов словаря, а производитель, бренд, тип, размер, контейнер и цена вычисляются детерминированным образом из комбинации случайных величин и ключа детали.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
func generate_part(partition_range, text_pool):
    name_random = RandomStringSequence(name_seed, NAME_WORDS, color_distribution)
    manufacturer_random = RandomBoundedInt(mfgr_seed, mfgr_min, mfgr_max)
    brand_random = RandomBoundedInt(brand_seed, brand_min, brand_max)
    type_random = RandomString(type_seed, part_type_distribution)
    size_random = RandomBoundedInt(size_seed, size_min, size_max)
    container_random = RandomString(container_seed, container_distribution)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)

    advance_all_randoms_to(partition_range.start_row)

    for row_id in range(partition_range.start_row, partition_range.end_row):
        part_key = row_id + 1
        mfgr_key = manufacturer_random.next_value()
        brand_key = mfgr_key * 10 + brand_random.next_value()

        emit PartRow(
            partkey = part_key,
            name = name_random.next_value(),
            mfgr = format("Manufacturer#%d", mfgr_key),
            brand = format("Brand#%d", brand_key),
            type = type_random.next_value(),
            size = size_random.next_value(),
            container = container_random.next_value(),
            retailprice = calculate_part_price(part_key) / 100.0,
            comment = comment_random.next_value()
        )

    finish_all_randoms_for_row()
```

Листинг 15 — Формирование таблицы part

2.5. Формирование таблицы partsupp

Ниже приведена символическая запись алгоритма формирования таблицы partsupp. Для каждой детали формируются четыре строки, соответствующие четырем поставщикам; ключ поставщика вычисляется детерминированным отображением от пары (partkey, supplier_number).

```
func generate_partsupp(partition_range, scale_factor, text_pool):
    availqty_random = RandomBoundedInt(availqty_seed, availqty_min, availqty_max, 4)
    supplycost_random = RandomBoundedInt(supplycost_seed, supplycost_min, supplycost_max, 4)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length, 4)

    advance_all_randoms_to(partition_range.start_row)

    for part_row in range(partition_range.start_row, partition_range.end_row):
        part_key = part_row + 1

        for supplier_number in range(0, 4):
            supplier_key = select_part_supplier(part_key, supplier_number, scale_factor)

            emit PartSuppRow(
                partkey = part_key,
                suppkey = supplier_key,
                availqty = availqty_random.next_value(),
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        supplycost = supplycost_random.next_value() / 100.0,
        comment = comment_random.next_value()
    )

    /* 4 поставщика на деталь */
    finish_all_randoms_for_row()

```

Листинг 16 — Формирование таблицы partsupp

2.6. Формирование таблицы customer

Ниже приведена символическая запись алгоритма формирования таблицы **customer**. Алгоритм близок к алгоритму формирования поставщиков, однако вместо специальных модификаций комментариев используется выбор рыночного сегмента из предопределенного распределения.

```

func generate_customer(partition_range, text_pool):
    address_random = RandomAlphaNumeric(address_seed, average_address_length)
    nation_random = RandomBoundedInt(nation_seed, 0, NATION_COUNT - 1)
    phone_random = RandomPhoneNumber(phone_seed)
    acctbal_random = RandomBoundedInt(acctbal_seed, acctbal_min, acctbal_max)
    segment_random = RandomString(segment_seed, market_segment_distribution)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length)

    advance_all_randoms_to(partition_range.start_row)

    for row_id in range(partition_range.start_row, partition_range.end_row):
        customer_key = row_id + 1
        nation_key = nation_random.next_value()

        emit CustomerRow(
            custkey = customer_key,
            name = format("Customer#%09d", customer_key),
            address = address_random.next_value(),
            nationkey = nation_key,
            phone = phone_random.next_value(nation_key),
            acctbal = acctbal_random.next_value() / 100.0,
            mktsegment = segment_random.next_value(),
            comment = comment_random.next_value()
        )

    finish_all_randoms_for_row()

```

Листинг 17 — Формирование таблицы customer

2.7. Формирование таблицы orders

Ниже приведена символическая запись алгоритма формирования таблицы **orders**. Ключ заказа строится по разреженной схеме ТРС-Н, ключ клиента выбирается с учетом пропуска “мертвых” клиентов, а итоговая стоимость и статус заказа вычисляются на основе моделирования всех строк будущей таблицы **lineitem** для данного заказа.

```

func generate_orders(partition_range, scale_factor, text_pool):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

order_date_random = RandomBoundedInt(order_date_seed, order_date_min, order_date_max)
line_count_random = RandomBoundedInt(line_count_seed, line_count_min, line_count_max)
customer_random = next_bounded_value(customer_seed, 1, max_customer_key(scale_factor),
scale_factor, 1)
priority_random = RandomString(priority_seed, order_priority_distribution)
clerk_random = RandomBoundedInt(clerk_seed, 1, max_clerk(scale_factor))
comment_random = RandomText(comment_seed, text_pool, average_comment_length)

quantity_random = RandomBoundedInt(quantity_seed, quantity_min, quantity_max, LINE_COUNT_MAX)
discount_random = RandomBoundedInt(discount_seed, discount_min, discount_max, LINE_COUNT_MAX)
tax_random = RandomBoundedInt(tax_seed, tax_min, tax_max, LINE_COUNT_MAX)
part_random = next_bounded_value(part_seed, 1, max_part_key(scale_factor), scale_factor,
LINE_COUNT_MAX)
ship_date_random = RandomBoundedInt(ship_date_seed, ship_date_min, ship_date_max,
LINE_COUNT_MAX)

advance_all_randoms_to(partition_range.start_row)

for row_id in range(partition_range.start_row, partition_range.end_row):
    order_index = row_id + 1
    order_key = make_sparse_order_key(order_index)
    order_date = order_date_random.next_value()
    customer_key = adjust_customer_key(customer_random.next_value(), max_customer_key(
scale_factor))

    total_price = 0
    shipped_count = 0
    line_count = line_count_random.next_value()

    for line_number in range(1, line_count + 1):
        quantity = quantity_random.next_value()
        discount = discount_random.next_value()
        tax = tax_random.next_value()
        part_key = part_random.next_value()
        ship_date = order_date + ship_date_random.next_value()

        total_price += calculate_line_total(part_key, quantity, discount, tax)
        if is_in_past(ship_date):
            shipped_count += 1

    emit OrderRow(
        orderkey = order_key,
        custkey = customer_key,
        orderstatus = derive_order_status(shipped_count, line_count),
        totalprice = total_price / 100.0,
        orderdate = order_date,
        orderpriority = priority_random.next_value(),
        clerk = format("Clerk#%09d", clerk_random.next_value()),
        shippriority = 0,
        comment = comment_random.next_value()
    )

    finish_all_randoms_for_row()

```

Листинг 18 — Формирование таблицы orders

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2.8. Формирование таблицы lineitem

Ниже приведена символическая запись алгоритма формирования таблицы lineitem. Партиционирование выполняется по диапазонам заказов; для каждого заказа регенерируются дата заказа и число позиций, после чего последовательно формируются строки поставок с согласованными ключами детали, поставщика и временными атрибутами.

```
func generate_lineitem(order_partition, scale_factor, text_pool):
    order_date_random = RandomBoundedInt(order_date_seed, order_date_min, order_date_max)
    line_count_random = RandomBoundedInt(line_count_seed, line_count_min, line_count_max)
    quantity_random = RandomBoundedInt(quantity_seed, quantity_min, quantity_max, LINE_COUNT_MAX)
    discount_random = RandomBoundedInt(discount_seed, discount_min, discount_max, LINE_COUNT_MAX)
    tax_random = RandomBoundedInt(tax_seed, tax_min, tax_max, LINE_COUNT_MAX)
    part_random = next_bounded_value(part_seed, 1, max_part_key(scale_factor), scale_factor,
    LINE_COUNT_MAX)
    supplier_number_random = RandomBoundedInt(supplier_number_seed, 0, 3, LINE_COUNT_MAX)
    ship_date_random = RandomBoundedInt(ship_date_seed, ship_date_min, ship_date_max,
    LINE_COUNT_MAX)
    commit_date_random = RandomBoundedInt(commit_date_seed, commit_date_min, commit_date_max,
    LINE_COUNT_MAX)
    receipt_date_random = RandomBoundedInt(receipt_date_seed, receipt_date_min, receipt_date_max,
    LINE_COUNT_MAX)
    return_flag_random = RandomString(return_flag_seed, return_flag_distribution, LINE_COUNT_MAX)
    ship_instruct_random = RandomString(ship_instruct_seed, ship_instruct_distribution,
    LINE_COUNT_MAX)
    ship_mode_random = RandomString(ship_mode_seed, ship_mode_distribution, LINE_COUNT_MAX)
    comment_random = RandomText(comment_seed, text_pool, average_comment_length, LINE_COUNT_MAX)

    advance_all_randoms_to(order_partition.start_row)

    for order_row in range(order_partition.start_row, order_partition.end_row):
        order_index = order_row + 1
        order_key = make_sparse_order_key(order_index)
        order_date = order_date_random.next_value()
        line_count = line_count_random.next_value()

        for line_number in range(1, line_count + 1):
            part_key = part_random.next_value()
            supplier_number = supplier_number_random.next_value()
            supplier_key = select_part_supplier(part_key, supplier_number, scale_factor)
            quantity = quantity_random.next_value()
            discount = discount_random.next_value()
            tax = tax_random.next_value()

            ship_date = order_date + ship_date_random.next_value()
            commit_date = order_date + commit_date_random.next_value()
            receipt_date = ship_date + receipt_date_random.next_value()

            emit LineitemRow(
                orderkey = order_key,
                partkey = part_key,
                suppkey = supplier_key,
                linenumber = line_number,
                quantity = quantity,
                extendedprice = calculate_part_price(part_key) * quantity / 100.0,
                discount = discount / 100.0,
                tax = tax / 100.0,
                returnflag = derive_return_flag(receipt_date, return_flag_random),
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
linestatus = derive_line_status(ship_date),  
shipdate = ship_date,  
commitdate = commit_date,  
receiptdate = receipt_date,  
shipinstruct = ship_instruct_random.next_value(),  
shipmode = ship_mode_random.next_value(),  
comment = comment_random.next_value()  
)  
  
finish_all_randoms_for_row()
```

Листинг 19 — Формирование таблицы lineitem

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. Символическое представление машинных кодов

Листинги, приведенные в предыдущих разделах настоящего документа, представлены в виде символической записи для компактного описания структуры и алгоритмов программы. В качестве основной формы представления используется символическая запись на исходном языке C++.

Подробное символическое представление машинных кодов в настоящий документ не включается, поскольку оно определяется целевой платформой, архитектурой процессора, версией компилятора, параметрами оптимизации и используемой средой компоновки.

Для типового варианта эксплуатации программы в среде GNU/Linux результатом сборки является исполняемый файл платформенного формата ELF. При этом развернутое представление объектного и машинного кода в настоящем документе не приводится.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.11.03-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Лист регистрации изменений

[illegible]